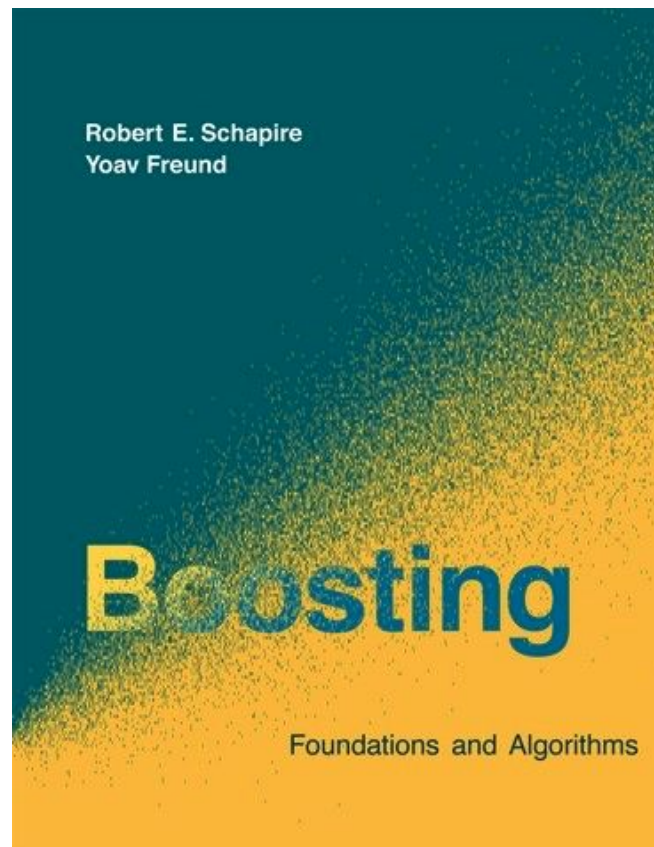


Boosting

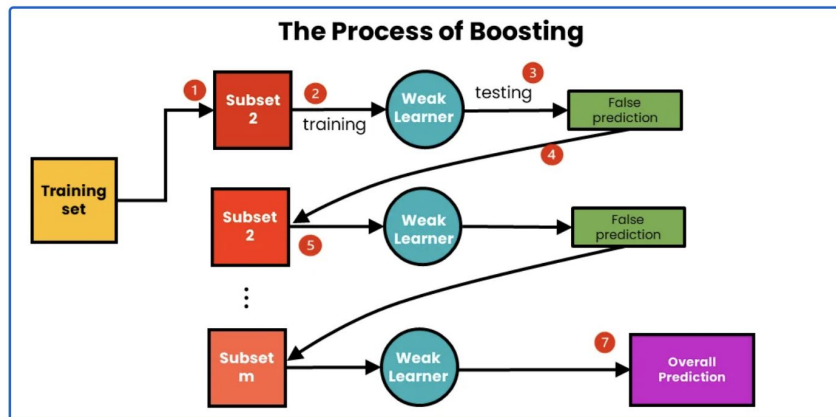
- boosting can help with High Bias models
- Adaboost
- Boosted trees in scikit learn
- XGboost
- CatBoost, LightGBM



What is Boosting?

- **Sequential ensemble method** that combines multiple "weak learners" into a strong learner
- Each new model focuses on correcting the errors made by previous models
- Models are added sequentially, with each attempting to reduce the residual error
- Weighted voting/averaging for final prediction
- Mathematically minimizes a loss function using gradient descent

"Boosting turns weak learners into strong learners by focusing on difficult examples"



Boosting vs. Bagging

Boosting	Bagging (e.g., Random Forest)
Sequential - models built one after another	Parallel - models built independently
Each model tries to correct errors of previous models	Each model tries to reduce overall error
Weighted models (later models may have higher influence)	Equal weight for all models
Reduces bias and variance	Primarily reduces variance
Prone to overfitting	More robust to overfitting
Can achieve lower error on training data	Better generalization with less tuning
Example: AdaBoost, Gradient Boosting, XGBoost	Example: Random Forest

Is Boosting Only for Decision Trees?

No! While decision trees are the most common weak learners, boosting can be applied to many algorithms:

- **Decision trees** - Most popular due to flexibility and interpretability
- **Linear models** - Boosted linear regression or logistic regression
- **Neural networks** - Boosted shallow neural nets
- **SVMs** - Boosted weak SVMs

Requirements for a weak learner:

- Better than random guessing
- Simple enough to avoid overfitting on its own
- Fast to train (especially important as we use many iterations)

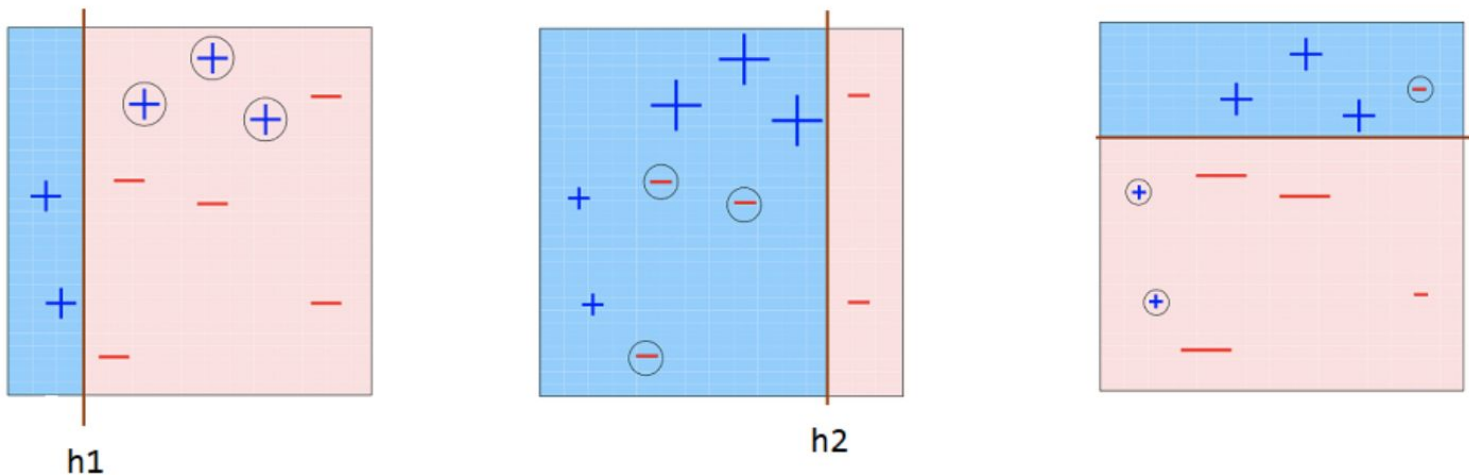
Note: Trees dominate in practice due to their natural handling of mixed data types, missing values, and interaction effects

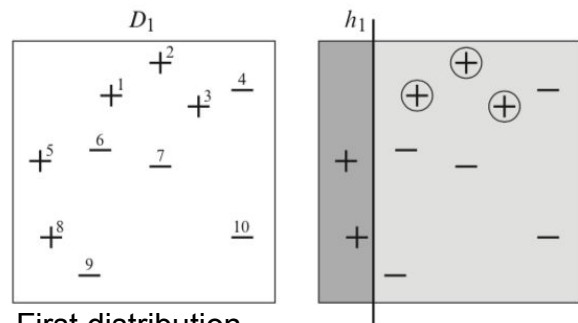
Adaboost

What is AdaBoost?

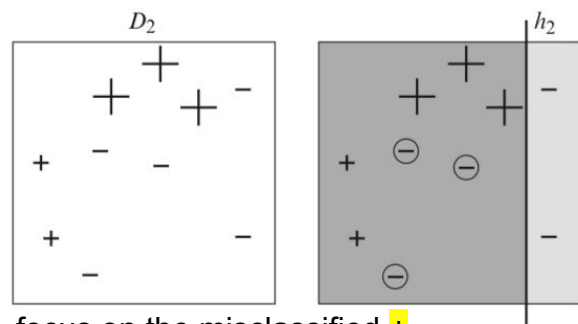
- **AdaBoost = Adaptive Boosting**
- First practical boosting algorithm (Freund & Schapire, 1996)
- Iteratively builds an ensemble by adding weak learners
- Focuses on **correctly classifying previously misclassified examples**
- Forms the foundation for many modern boosting algorithms

"AdaBoost is the best off-the-shelf classifier in the world" — Leo Breiman

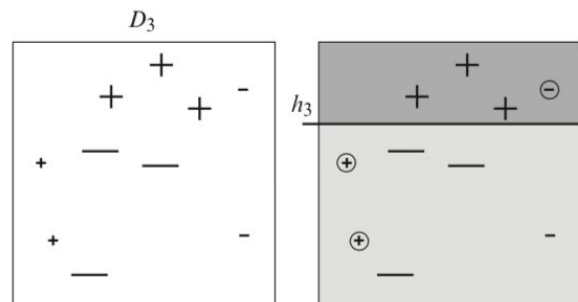




First distribution



focus on the misclassified +



focus on the misclassified -

$$H = \text{sign} \left(0.42 \begin{array}{|c|} \hline \text{shaded} \\ \hline \end{array} + 0.65 \begin{array}{|c|} \hline \text{shaded} \\ \hline \end{array} + 0.92 \begin{array}{|c|} \hline \text{shaded} \\ \hline \end{array} \right)$$

$$= \begin{array}{|c|c|c|} \hline \text{shaded} & \text{shaded} & \text{shaded} \\ \hline \text{shaded} & \text{shaded} & \text{shaded} \\ \hline \text{shaded} & \text{shaded} & \text{shaded} \\ \hline \end{array}$$

Figure 1.2

The combined classifier for the toy example of figure 1.1 is computed as the sign of the weighted sum of the three weak hypotheses, $\alpha_1 h_1 + \alpha_2 h_2 + \alpha_3 h_3$, as shown at the top. This is equivalent to the classifier shown at the bottom. (As in figure 1.1, the regions that a classifier predicts positive are indicated using darker shading.)

Additive models

An additive model builds a prediction by iteratively adding simple components together. Each component tries to explain some part of the target that hasn't been captured yet.

Mathematically: $F(x) = F_0(x) + F_1(x) + F_2(x) + \dots + F_M(x)$

Both AdaBoost and gradient boosting are additive models:

- AdaBoost: adds weighted weak learners
- Gradient Boosting: adds trees that predict residuals

The key advantage is that complex functions can be approximated by combining simple components.

How AdaBoost works?

Initialize sample weights uniformly (each sample weight = $1/n$)

For each iteration $t = 1, 2, \dots, T$:

- Train a weak learner h_t using the current weights
- Calculate the weighted error ϵ_t
- Compute model weight $\alpha_t = 0.5 * \ln((1-\epsilon_t)/\epsilon_t)$
- **Update sample weights:**
 - Increase weights of misclassified samples
 - Decrease weights of correctly classified samples
- Normalize weights to sum to 1

Final prediction: weighted majority vote of all weak learners

How AdaBoost works?

Equal initial weights

First classifier trained on weighted data

Misclassified points get higher weights

Second classifier focuses more on previously misclassified points

Process repeats until ensemble is built

Final boundary combines all weak classifiers

Adaboost Strengths & Weaknesses

AdaBoost Strengths

- **Simple to implement** and understand
- **Less prone to overfitting** than other boosting methods (though not immune)
- Can **identify outliers** (samples with consistently high weights)
- **Feature selection** comes naturally (important features used more frequently)
- **No need for preprocessing** like scaling features
- Can use **various base learners** (not just decision trees)
- Particularly strong for **binary classification** problems

AdaBoost Limitations

- **Sensitive to noisy data and outliers**
- Can **underperform** if weak learners are too complex
- **Slower training** compared to bagging methods
- Base learners must be **better than random guessing**
- Less flexible than modern boosting algorithms (XGBoost, LightGBM)
- May require large number of estimators for complex problems
- Less effective for **highly imbalanced datasets**

```

def adaboost(X, y, num_estimators=50):
    n_samples = len(X)
    weights = np.ones(n_samples) / n_samples # Initialize weights

    estimators = []
    alphas = []

    for _ in range(num_estimators):
        # Train weak learner with current weights
        estimator = DecisionTreeClassifier(max_depth=1)
        estimator.fit(X, y, sample_weight=weights)
        estimators.append(estimator)

        # Predict and calculate weighted error
        predictions = estimator.predict(X)
        incorrect = (predictions != y)
        error = np.sum(weights * incorrect) / np.sum(weights)

        # Calculate estimator weight (alpha)
        alpha = 0.5 * np.log((1 - error) / (error + 1e-10))
        alphas.append(alpha)

        # Update weights
        weights *= np.exp(-alpha * y * predictions)
        weights /= np.sum(weights) # Normalize

    return estimators, alphas

```

Demo Adaboost

<https://colab.research.google.com/drive/1oYhtqPLc1rz0nwwYkEF1M-BYBouonB3d>

Key Parameters in scikit-learn's AdaBoostClassifier

AdaBoostClassifier

```
class sklearn.ensemble.AdaBoostClassifier(estimator=None, *,  
n_estimators=50, learning_rate=1.0, algorithm='deprecated', random_state=None)
```

[\[source\]](#)

An AdaBoost classifier.

An AdaBoost [\[1\]](#) classifier is a meta-estimator that begins by fitting a classifier on the original dataset and then fits additional copies of the classifier on the same dataset but where the weights of incorrectly classified instances are adjusted such that subsequent classifiers focus more on difficult cases.

- **base_estimator**: The weak learner (default: DecisionTreeClassifier with max_depth=1)
- **n_estimators**: Number of weak learners (default: 50)
- **learning_rate**: Contribution of each classifier (default: 1.0)
- **algorithm**: Type of AdaBoost algorithm:
 - 'SAMME': Original discrete AdaBoost algorithm
 - 'SAMME.R': Uses real-valued predictions, usually better
- **random_state**: Controls randomness for reproducibility

Gradient boosting

Main Parameters of Gradient Boosting

Learning Algorithm Parameters

- **learning_rate**: Shrinks contribution of each tree (smaller values need more trees)
- **n_estimators**: Number of sequential trees to be built
- **subsample**: Fraction of samples used for fitting individual trees (< 1.0 enables stochastic boosting)
- **loss function**: Determines the optimization goal (e.g., 'squared_error', 'absolute_error')

Tree-Specific Parameters

- **max_depth**: Maximum depth of each decision tree
- **min_samples_split**: Minimum samples required to split a node
- **min_samples_leaf**: Minimum samples required at a leaf node
- **max_features**: Number of features to consider for best split

Tuning Strategy: Trees or Gradient First?

Recommended Approach: Two-Stage Tuning

1. First: Fix gradient parameters, tune tree structure

- Set moderate learning_rate (0.1) and high n_estimators (1000)
- Focus on controlling tree complexity:
 - Tune max_depth (typically 3-8)
 - Adjust min_samples_split and min_samples_leaf
 - Consider max_features if dimensionality is high
- Goal: Find right tree complexity that balances underfitting/overfitting

2. Second: Fix tree structure, tune gradient parameters

- Using optimal tree structure from step 1
- Tune learning_rate (try smaller values: 0.01-0.1)
- Adjust n_estimators accordingly (lower learning_rate requires more trees)
- Experiment with subsample for stochastic boosting (0.5-0.8)
- Fine-tune the loss function if needed

[How to Configure the Gradient Boosting Algorithm](#)

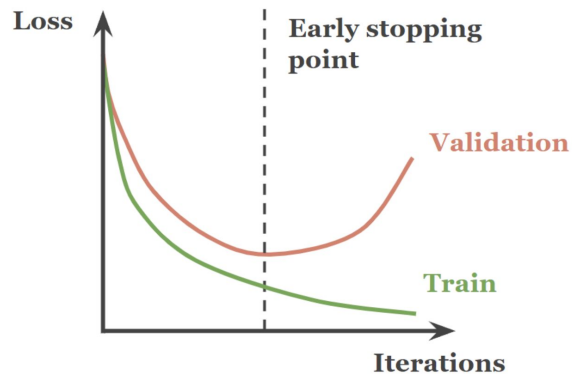
AdaBoost vs. Gradient Boosting

AdaBoost	Gradient Boosting
Updates sample weights	Updates residual errors
Typically uses decision stumps (one-level trees)	Typically uses deeper trees
Minimizes exponential loss	Can use various loss functions
More sensitive to noisy data and outliers	Can be more robust depending on loss function
Usually requires fewer regularization parameters	Offers more tuning options for regularization
Often needs more iterations	May achieve good results with fewer iterations

Early Stopping and Pruning

- **Early stopping:** Stop adding trees when validation error stops improving
 - Set `n_estimators` high and use `early_stopping_rounds` in modern implementations
 - Prevents overfitting and saves computation time
- **Pruning:** Retrospectively remove trees that contribute to overfitting
 - Less common in gradient boosting than early stopping
 - Can be implemented manually by evaluating model with different numbers of trees

Monitoring validation curves during training is crucial for both approaches



Summary: Keys to Successful Gradient Boosting

- Start with shallow trees (`max_depth=3`) and moderate learning rate (0.1)
- Use cross-validation for all parameter tuning
- Tune tree parameters first, then gradient parameters
- Watch for overfitting with validation data
- Remember that a well-tuned simpler model often outperforms poorly tuned complex ones
- Modern implementations (XGBoost, LightGBM, CatBoost) offer additional optimizations and parameter options

Gradient Boosting in sklearn

GradientBoostingClassifier

```
class sklearn.ensemble.GradientBoostingClassifier(*, loss='log_loss',
learning_rate=0.1, n_estimators=100, subsample=1.0, criterion='friedman_mse',
min_samples_split=2, min_samples_leaf=1, min_weight_fraction_leaf=0.0,
max_depth=3, min_impurity_decrease=0.0, init=None, random_state=None,
max_features=None, verbose=0, max_leaf_nodes=None, warm_start=False,
validation_fraction=0.1, n_iter_no_change=None, tol=0.0001, ccp_alpha=0.0)
```

Gradient Boosting for classification.

[\[source\]](#)

This algorithm builds an additive model in a forward stage-wise fashion; it allows for the optimization of arbitrary differentiable loss functions. In each stage `n_classes_` regression trees are fit on the negative gradient of the loss function, e.g. binary or multiclass log loss. Binary classification is a special case where only a single regression tree is induced.

[HistGradientBoostingClassifier](#) is a much faster variant of this algorithm for intermediate and large datasets (`n_samples >= 10_000`) and supports monotonic constraints.

Read more in the [User Guide](#).

<https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.GradientBoostingClassifier.html>

GradientBoostingClassifier

- **loss**: deviance: 'deviance' refers to logistic regression for classification with probabilistic outputs
- **learning_rate**: learning rate shrinks the contribution of each tree by `learning_rate`.
- **n_estimators**: number of stages, epoch, trees
- **subsample**: Choosing `subsample < 1.0` leads to a reduction of variance and an increase in bias. (Stochastic Gradient Boosting)
- Trees regularization:
 - **max_depth**: The maximum depth of the individual regression estimators
 - **min_samples_split**: The minimum number of samples required to split an internal node
 - **min_samples_leaf**: The minimum number of samples required to be at a leaf node
- **max_features**: The number of features to consider when looking for the best split

Documentation

- [3.2.4.3.5. sklearn.ensemble.GradientBoostingClassifier — scikit-learn 0.23.2 documentation](#)
- [1.11.4. Gradient Tree Boosting](#)

Demo Gradient Boosting Regressor

- <https://colab.research.google.com/drive/1RjPnUThGJ306eWOzOD-FIPJ-abMV2RXI#scrollTo=pecFVZc2CvvI>

dmlc
XGBoost

<https://xgboost.readthedocs.io/en/stable/>



Bojan Tunguz  
@tunguz

XGBoost is all you need.

XGboost vs gradient boosting in sklearn

Overview

- Both implement gradient boosting algorithms
- Same core concept: sequential ensemble of trees
- Different implementations with different strengths
- Important to understand differences for optimal model selection

Performance & Speed

XGBoost

- Significantly faster (can be 10x faster on large datasets)
- Optimized parallel processing
- Better memory management
- Specialized data structures ("blocks") for efficient computation

Scikit-learn

- Sequential processing
- Less optimized for very large datasets
- More focused on API consistency than raw performance

XGboost vs gradient boosting in sklearn

Feature	Scikit-learn GradientBoosting	XGBoost
Built-in cross-validation	No (use GridSearchCV)	Yes (xgb.cv)
Early stopping	No	Yes
Continue training	No	Yes
Missing value handling	No (preprocess required)	Yes (automatic)
GPU acceleration	No	Yes
Distributed computing	No	Yes

XGboost vs gradient boosting in sklearn

Regularization Options

XGBoost

- L1 (lasso) regularization on leaf weights
- L2 (ridge) regularization on leaf weights
- Gamma parameter (minimum loss reduction for split)
- Column subsampling (like Random Forest)
- Dropout-like functionality ("dart" booster)

Scikit-learn

- Limited to tree-level complexity control
- Basic subsample parameter
- No specialized weight regularization

XGboost vs gradient boosting in sklearn

Parameter Comparison

Purpose	Scikit-learn Parameter	XGBoost Parameter
Learning rate	<code>learning_rate</code>	<code>eta</code>
Number of trees	<code>n_estimators</code>	<code>num_boost_round</code>
Tree depth	<code>max_depth</code>	<code>max_depth</code>
Min samples to split	<code>min_samples_split</code>	<code>min_child_weight</code>
Row sampling	<code>subsample</code>	<code>subsample</code>
Column sampling	Not available	<code>colsample_bytree</code>

XGboost vs gradient boosting in sklearn

```
from sklearn.ensemble import  
GradientBoostingClassifier  
from sklearn.pipeline import Pipeline  
from sklearn.preprocessing import  
StandardScaler  
  
pipe = Pipeline([  
    ('scaler', StandardScaler()),  
    ('gb', GradientBoostingClassifier())  
])
```

```
import xgboost as xgb  
  
from sklearn.pipeline import Pipeline  
from sklearn.preprocessing import  
StandardScaler  
  
pipe = Pipeline([  
    ('scaler', StandardScaler()),  
    ('xgb', xgb.XGBClassifier())  
])
```

Ecosystem Considerations

XGBoost is part of a family of specialized gradient boosting libraries:

- **LightGBM**: Even faster, uses histogram-based splits
- **CatBoost**: Better handling of categorical features

Scikit-learn focuses on API consistency:

- Same interface as other models
- Easier integration with scikit-learn ecosystem
- Simpler hyperparameter naming conventions

Other flavors of XGBoost



XGBoost vs. LightGBM vs. CatBoost

- **XGBoost**: The original game-changer (2014)
- **LightGBM**: Microsoft's speed-focused alternative (2017)
- **CatBoost**: Yandex's categorical-feature specialist (2017)



XGBoost

- Pre-sorted algorithm & histogram-based algorithm
- Sparsity-aware split finding
- Cache-aware access patterns
- Blocks for out-of-core computation

<https://github.com/dmlc/xgboost>



LightGBM

- Gradient-based One-Side Sampling (GOSS)
- Exclusive Feature Bundling (EFB)
- Leaf-wise tree growth vs. level-wise

<https://lightgbm.readthedocs.io/>



CatBoost

- Ordered Boosting (reduces prediction shift)
- Automated categorical encoding
- Symmetric trees with improved leaf value estimation
- Native feature importance calculation

<https://github.com/catboost/>

XGBoost vs. LightGBM vs. CatBoost

Performance & Speed

Framework	Training Speed	Memory Usage	Inference Speed
XGBoost	Fast	Moderate	Fast
LightGBM	Very Fast	Low	Fast
CatBoost	Moderate (slower for training)	Moderate-High	Very Fast

XGBoost vs. LightGBM vs. CatBoost

Categorical Feature Handling

XGBoost

- No native categorical support
- Requires pre-processing (one-hot encoding)
- Can be inefficient with high-cardinality features

LightGBM

- Native categorical support
- Uses special algorithm for splits on categorical features
- Limited to 256 unique categories per feature

CatBoost

- **Best-in-class categorical handling**
- Automatic ordered target statistics
- Handles high-cardinality features efficiently
- Target encoding with built-in safeguards against leakage

XGBoost vs. LightGBM vs. CatBoost

Summary: When to Choose Each Framework

Framework	Best For	Consider Alternatives When
XGBoost	Production environments, balanced performance, general-purpose use	Speed is critical, many categorical features
LightGBM	Very large datasets, speed-critical applications, limited memory	Many categorical features, need robust defaults
CatBoost	Categorical-heavy data, need minimal tuning, prediction stability	Extremely large datasets, memory constraints

Benchmarks

- Kaggle Notebook [Battle of the Boosting Algos: LGB, XGB, Catboost](#)
- [Battle of the Boosting Algorithms](#)
- [CatBoost vs. Light GBM vs. XGBoost | by Alvira Swalin](#)
- [Lightgbm vs xgboost vs catboost](#)
- [XGBoost, LightGBM or CatBoost — which boosting algorithm should I use?](#)
- [CatBoost vs. Light GBM vs. XGBoost](#)

Posts (probably no longer valid)

- https://duchesnay.github.io/pystatsml/machine_learning/ensemble_learning.html#boosting
- slides https://docs.google.com/presentation/d/14cbd9SHTVhDfzt9JCO8Eproo3GYpESTvpkz1NFJZYvE/edit#slide=id.g9c7268ca97_0_29
- <https://machinelearningmastery.com/gentle-introduction-gradient-boosting-algorithm-machine-learning/>
- <https://machinelearningmastery.com/boosting-and-adaboost-for-machine-learning/>
- series <https://www.xgblog.ai/p/xgboost-is-all-you-need-part-3-gradient>
- slides pdf https://www.chengli.io/tutorials/gradient_boosting.pdf
- <https://www.sciencedirect.com/topics/engineering/adaboost>